

STRATA: Learning Strategy Representations for Interaction-Informed Retries

Anonymous

Abstract—Manipulation tasks often admit several strategies, but hidden physical properties can make the right choice uncertain. Successful demonstrations show how to carry out these strategies, yet a robot must also maintain a strategy throughout an attempt and use interaction to choose what to try next. We introduce STRATA, which uses a shared strategy representation for execution within attempts and selection across attempts. We jointly train a strategy encoder, codebooks, and a strategy-conditioned flow-matching policy on successful demonstrations. Two-stage residual vector quantization maps each complete demonstration to a strategy code. The code stays fixed throughout an attempt while the flow-matching policy responds to current observations. We then freeze the execution policy and codebooks and train a high-level policy with reinforcement learning to select codes from completed attempts’ observation–action histories. This focuses exploration on sequences of strategy choices. In simulated box opening across 20 physical conditions, STRATA achieves 92.7% success within three attempts, compared with 57.6% for history-conditioned, per-attempt DSRL. Representations with a discrete component support strong learned selection in this setting. Zero-shot deployment on a real robot across four physical conditions yields 70% success within three attempts, compared with 55% for per-attempt DSRL. Website: <https://strata-icra27.github.io/>

I. INTRODUCTION

Robots often face manipulation tasks that admit several distinct ways of interacting. For example, when opening a box, both arms can be used to pull a heavy lid, one can pull the lid open while the other is placed on back to avoid the box to slip, or both can be placed at different parts of the box to force a rusty joint open, followed by distinct manipulation trajectories (see Fig. 1, top left). We call these discrete groups of trajectories carrying out an attempt *manipulation strategies*. Which strategy succeeds depends thus on physical properties that are not be observable before interaction, such as friction, mass distribution, compliance, or articulation. Visually similar objects may therefore admit different strategies to be executed, while only a subset of them will achieve success.

Modern diffusion and flow-matching policies are able to capture such a multi-modal space of trajectories and make it available to the robot [1,2]. But having several strategies available does not tell the robot which one will work: when the relevant physical properties are hidden, the current observation may not distinguish between them and the demonstrations of successful strategies do not directly supervise which strategy to choose after a failed attempt. Some methods collect failure-and-recovery demonstrations [3]; others learn from successful demonstrations and guide retries away from previous failures [4,5]. History-conditioned reinforcement learning can instead learn from task outcomes, but learning

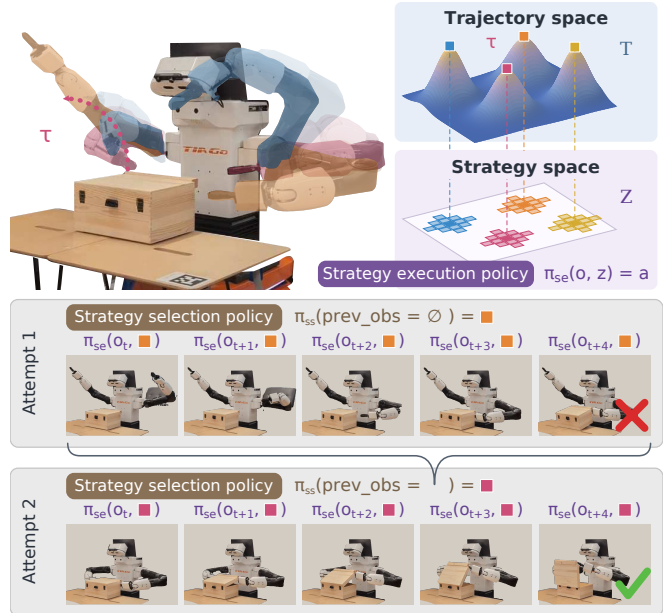


Fig. 1. Multimodal demonstrations show different ways to achieve the same goal. In box opening, some use one arm to brace and the other to lift, whereas others lift with both arms. Which strategy succeeds depends on physical properties that may only become apparent through interaction. The demonstrated alternatives can therefore shape what the robot explores when it does not yet know which will work. STRATA learns discrete strategy codes $z \in \mathcal{Z}$ from complete demonstrated trajectories $\tau \in \mathcal{T}$, jointly with a flow-matching strategy execution policy π_{se} . The strategy selection policy π_{ss} chooses a code before each attempt using the recorded interaction history, labeled *prev_obs*. This history is empty before Attempt 1. The selected code stays fixed throughout an attempt while $\pi_{se}(o_t, z)$ generates actions from current observations. The observed robot–object response then informs the next strategy choice.

execution and strategy selection together requires exploring long sequences of low-level actions: when an attempt fails, the reward alone does not identify whether the strategy was unsuitable or its execution was poor, complicating credit assignment. Steering a pretrained policy through its initial noise reduces this burden [6], but leaves the strategy expressed across an attempt implicit; which does not ensure that the policy follows a consistent strategy throughout the attempt, leading to strategy switches and failures [7].

Our insight is to use the same strategy representation to maintain consistent behavior within an attempt and support strategy selection across attempts. We introduce **STRATA**, a framework that jointly learns a discrete strategy representation and a strategy-conditioned flow-matching policy from

successful demonstrations (Fig. 1). Each code represents the strategy expressed in a complete demonstration and remains fixed throughout an attempt, while the flow-matching policy responds to current observations. We then freeze the execution policy and train a high-level policy with reinforcement learning to choose a strategy from completed attempts’ observation–action histories. This focuses exploration and credit assignment on a few strategy choices across attempts, using task rewards to learn which strategy to try next.

We evaluate **STRATA** on box opening, where hidden physical properties affect which strategy succeeds. Across 20 physical conditions in simulation, **STRATA** succeeds within three attempts in 92.7% of trials, compared with 57.6% for history-conditioned, per-attempt DSRL. Our representation study shows that a controller can execute useful strategies even when the high-level policy struggles to select them. The history ablation shows that observing how an attempt unfolds improves later choices beyond knowing which strategies failed. We also deploy the policies on a real robot without further training. Across four physical conditions, **STRATA** achieves 70% success within three attempts, compared with 55% for per-attempt DSRL.

II. RELATED WORK

Multimodal visuomotor policies. Diffusion and flow-matching policies learn distributions over demonstrated action sequences [1,2]. VQ-BeT applies residual vector quantization to actions or action chunks and predicts their codes from observations [8]. PF-DAG separates discrete action-chunk modes from continuous generation to reduce unwanted switching [7], while VQActFlow generates quantized action-chunk codes with flow matching [9]. Our distinction is the scope and use of the representation: we encode a complete demonstration, jointly train a reactive controller to use that code across action windows, and make it the action of a history-conditioned policy that selects once per attempt.

Skill learning and hierarchical policies. Learned skills give high-level policies reusable behaviors for composing task solutions [10–16]. SIMPL also learns skills from offline trajectories and uses interaction context to guide a meta-trained high-level policy [17]. Its skills capture shorter behaviors that the policy composes into long-horizon solutions. In our setting, demonstrations provide complete alternatives for the same goal: the high-level policy chooses which alternative to try, rather than which skills to assemble within a solution. DSRL and FRS also steer pretrained generative policies, but choose initial noise rather than codes learned from complete demonstrations [6,18].

Learning from interaction. Interactive perception uses action to acquire informative observations [19,20] and reveal information unavailable from passive observation [21,22], and input design for system identification seeks informative actions for estimating physical parameters [23]. History-conditioned imitation learns responses from demonstrated interactions [24–26]; other approaches use recovery demonstrations [3] or annotated failure feedback [27]. RL can

learn feedback-dependent control and exploration [28,29], while meta-RL learns how experience should guide later decisions [30,31].

Several methods already guide retries without corrective demonstrations. Bellman-Guided Retrials learns a value function from successful demonstrations to detect poor progress, trigger recovery, and bias subsequent action sampling away from past mistakes [4]. CCDP uses specified failure features to guide diffusion sampling toward alternatives, also learning from successful demonstrations [5]. HAVE uses interaction history to score proposals from a generative policy [32], while FAR updates policy weights from failures [33]. Rather than scoring or steering generated proposals, **STRATA** learns strategy codes jointly with the executor and trains a policy to select these whole-attempt alternatives from observation–action histories. Its reward covers the remaining attempts in a trial, so a choice can contribute through immediate success or through information useful for a later choice. This requires simulator interaction during training, but neither corrective demonstrations nor deployment-time weight updates.

III. METHOD

STRATA uses a shared strategy representation for execution and selection. The strategy execution policy π_{se} uses a strategy code z throughout an attempt, while the strategy selection policy π_{ss} chooses that code from previous interactions. We first learn the strategy representation and π_{se} from successful demonstrations. We then freeze the execution components and train π_{ss} through interaction (Fig. 2).

A. Problem Setup

Training assumes a dataset of successful demonstrations and a simulator for interaction. A trial contains up to K manipulation attempts. We use $K = 3$ in training and evaluation. The object’s physical properties stay fixed within a trial and are not supplied explicitly to the policies. Between attempts, we reset the object to a new random pose while keeping these properties unchanged. The recorded interaction history carries forward to guide the next strategy choice.

Each attempt has a horizon of $T = 156$ control steps at 5 Hz. At step t of attempt k , the robot observes $o_{k,t} = (P_{k,t}, q_{k,t})$, containing a point cloud and robot state, and issues a command $a_{k,t}$. Commands have 16 components: 14 arm-joint displacements and two gripper commands, executed by joint-position PD control. We record the attempt as

$$\tau_k = ((o_{k,0}, a_{k,0}), (o_{k,1}, a_{k,1}), \dots, (o_{k,T-1}, a_{k,T-1})), \quad (1)$$

where each observation precedes its paired command. Before attempt k , the strategy selection policy uses the completed histories $H_k = (\tau_1, \dots, \tau_{k-1})$ to select a strategy code z_k . The strategy execution policy uses z_k throughout the attempt while predicting short command windows from current observations. Success is assessed at the end of the attempt. The trial ends after success or after all K attempts have been used.

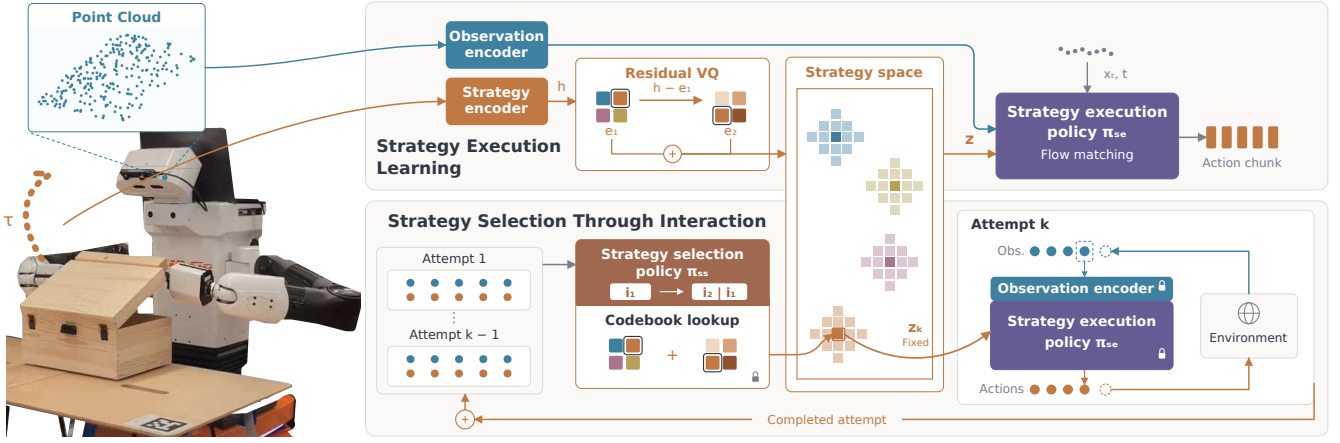


Fig. 2. **Method overview.** **Top:** We jointly train the strategy encoder, residual VQ codebooks, observation encoder, and strategy execution policy π_{se} on successful demonstrations. The strategy encoder maps each complete demonstrated trajectory τ to a strategy code z , obtained from two-stage residual VQ. The same code conditions π_{se} throughout the demonstration, while current observations determine the action chunks. **Bottom:** We freeze the codebooks, observation encoder, and π_{se} , and train the strategy selection policy π_{ss} from completed attempts. Given the observation–action histories of previous attempts, π_{ss} selects two codebook indices, with the second conditioned on the first. Their entries form z_k , which remains fixed during attempt k while π_{se} reacts to current observations. The completed attempt is then added to the history for the next strategy choice.

B. Learning Strategies and Their Execution

We first learn a strategy representation together with the strategy execution policy π_{se} . The strategy encoder reads the complete action sequence of a successful demonstration, represented as absolute joint targets, without strategy annotations. A temporal convolutional network processes this sequence and produces a 64-dimensional embedding.

Let U_τ denote the normalized sequence of absolute joint targets from demonstration τ , and let $h = E_{str}(U_\tau)$. Two-stage residual VQ approximates h with entries from codebooks $\mathcal{C}^{(1)}$ and $\mathcal{C}^{(2)}$, each containing 32 vectors of dimension 64:

$$\begin{aligned} i_1 &= \arg \min_i \|h - e_i^{(1)}\|_2^2, \\ i_2 &= \arg \min_j \|h - e_{i_1}^{(1)} - e_j^{(2)}\|_2^2, \\ z_\tau &= e_{i_1}^{(1)} + e_{i_2}^{(2)}. \end{aligned} \quad (2)$$

The first entry approximates the encoder output, and the second approximates the remaining residual. Both choices are discrete. Every action window from the demonstration uses the same strategy code z_τ .

The strategy execution policy π_{se} is a conditional flow-matching policy with a temporal U-Net. It predicts eight commands and executes the first command before replanning. Its observation contains 256 foreground XYZ points in world coordinates and 18 arm and finger joint positions. Simulation segmentation retains the robot and object. A PointNet-style encoder and a joint-state multilayer perceptron each produce 64 features, which form $c_t = E_{obs}(o_t)$.

Let A_t denote the normalized window of demonstrated commands $(a_t, \dots, a_{t+7})$. To train π_{se} [2], we sample Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ and flow time $s \sim \mathcal{U}[0, 1]$, then form

$$x_s = (1 - s)\epsilon + sA_t. \quad (3)$$

The policy predicts the velocity $A_t - \epsilon$ with the loss

$$\mathcal{L}_{FM} = \mathbb{E}_{\tau, t, \epsilon, s} [\text{MSE}(v_\theta(x_s, s, c_t, z_\tau), A_t - \epsilon)]. \quad (4)$$

Here, s indexes the noise-to-action transformation, while t indexes robot interaction. At inference, we integrate from Gaussian noise using 10 Euler steps.

We jointly train the strategy encoder, codebooks, observation encoder, and π_{se} with

$$\mathcal{L} = \mathcal{L}_{FM} + \mathcal{L}_{cb}^{(1)} + \mathcal{L}_{cb}^{(2)} + \lambda_{com}\mathcal{L}_{com}. \quad (5)$$

The first codebook loss moves the selected entry toward the encoder output, and the second moves its selected entry toward the residual left by the first. The commitment loss keeps the encoder output close to their sum; we use $\lambda_{com} = 0.25$. A straight-through estimator passes the flow-matching gradient to the strategy encoder, encouraging it to preserve information needed to predict the demonstrated commands. The codebooks receive gradients from their codebook losses, not from the flow-matching loss.

C. Learning Strategy Selection from Interaction

After learning strategy execution, we freeze π_{se} , its observation encoder, and both codebooks. The strategy encoder is used only during demonstration training. During RL training and deployment, the strategy selection policy π_{ss} selects codebook indices directly.

The strategy selection policy summarizes each completed attempt using temporal convolutions and attention pooling over observation–action features. Self-attention with learned attempt-position embeddings combines these summaries across attempts. For the first attempt, $H_1 = \emptyset$: π_{ss} uses a learned empty-history embedding and receives no current observation.

The policy first selects one codebook index and then selects the second conditioned on the first:

$$\begin{aligned} \pi_{ss,\psi}(i_{k,1}, i_{k,2} \mid H_k) \\ = \pi_{ss,\psi,1}(i_{k,1} \mid H_k) \pi_{ss,\psi,2}(i_{k,2} \mid i_{k,1}, H_k). \end{aligned} \quad (6)$$

The selected entries sum to z_k . Both indices are chosen before the attempt, and z_k remains fixed throughout its execution. After the attempt ends, its recorded observation–action history is appended to H_k for the next strategy choice.

We train π_{ss} with PPO, treating each complete attempt as one transition. The return for a trial is

$$R = 10S - 0.1N, \quad (7)$$

where S indicates trial success and $N \leq K$ is the number of attempts used. Success receives a reward of 10, and each attempt costs 0.1. We use no discounting across attempts ($\gamma = 1$). Because the return includes later attempts, a strategy choice can contribute either by succeeding immediately or by producing interaction that helps inform a later choice. The main strategy selection policy trains for 200,000 simulated trials using Adam with a learning rate of 3×10^{-4} .

IV. EXPERIMENTS

We organize the experiments around four research questions:

- 1) Does **STRATA** help the robot select better strategies through interaction?
- 2) What makes a strategy representation suitable for execution and selection?
- 3) How much information from previous attempts does strategy selection need?
- 4) Does strategy selection learned in simulation transfer zero-shot to a real robot?

A. Task and Evaluation

We study box opening under the same 20 physical conditions in training and evaluation. We vary box density, table-surface and gripper friction, and hinge friction and damping, with additional variation in object geometry. These properties change the contact, support, and lifting actions needed to open the box. Low box mass or table friction can make bracing useful, whereas a heavy lid may require both arms to lift. The policies receive point clouds and proprioception, but no explicit physical-parameter values.

Each trial allows up to three attempts. After a failed attempt, we close the lid and place the box at a new random pose while keeping all physical parameters unchanged and retaining the interaction history. An attempt succeeds if the lid opens beyond 80° and remains open after the grippers withdraw, without the box falling off the table or the robot colliding with the table. We assess success at the end of the attempt. We report the *success rate within k attempts* (Success@ k), the fraction of trials that succeed by attempt k , with Success@3 as the primary metric.

We collect 19,553 successful demonstrations by executing 50 source motions across the physical conditions and retaining successful, valid rollouts. We use 17,569 demonstrations for

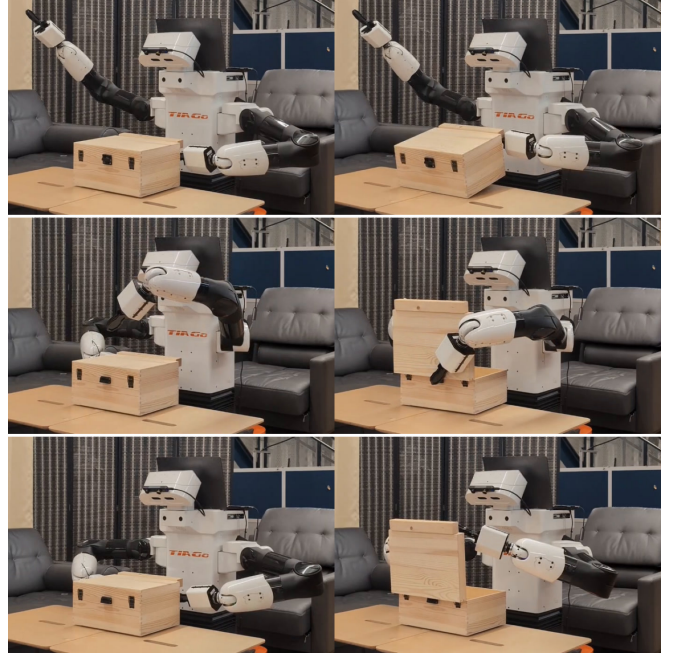


Fig. 3. **Real-world execution under different physical conditions.** Each row shows a strategy near contact (left) and its outcome (right). Top: a single-arm lift raises the box body rather than opening the lid on a stiff-hinge box. Middle: one arm contacts the lid top while the other lifts from the front, opening the same box. Bottom: both arms lift together to open a box with a heavy lid. The examples illustrate why the physical condition matters when choosing a strategy.

training and 1,984 for validation, with condition-balanced sampling and no strategy annotations. Simulation results average two seeds for both the flow-matching and RL policies. We evaluate 100 trials per condition and seed and average equally across the 20 conditions.

B. Strategy Selection through Interaction

We compare **STRATA** with a per-step strategy baseline inspired by RMA’s architecture [28] (RMA-like) and two history-conditioned DSRL variants [6]. RMA-like uses our strategy-conditioned flow-matching policy but predicts its strategy at each replanning step rather than once per attempt; it does not estimate physical parameters. Both DSRL variants steer an unconditioned flow-matching policy through its initial noise. Per-step DSRL (DSRL-PS) selects new noise at each replanning step. Per-attempt DSRL (DSRL-PA) selects one noise tensor and reuses it throughout the attempt. All execution policies continue to receive current observations and predict new command windows.

Figure 4 shows success across attempts. **STRATA** reaches 92.7% Success@3, compared with 57.6% for per-attempt DSRL, a gain of 35.1 percentage points. Both methods make one high-level choice per attempt, but **STRATA** chooses a strategy code learned jointly with the controller from complete demonstrations. The code conditions successive action predictions, whereas DSRL’s initial noise leaves the relation between these predictions implicit. The comparison supports learning a shared representation for execution and

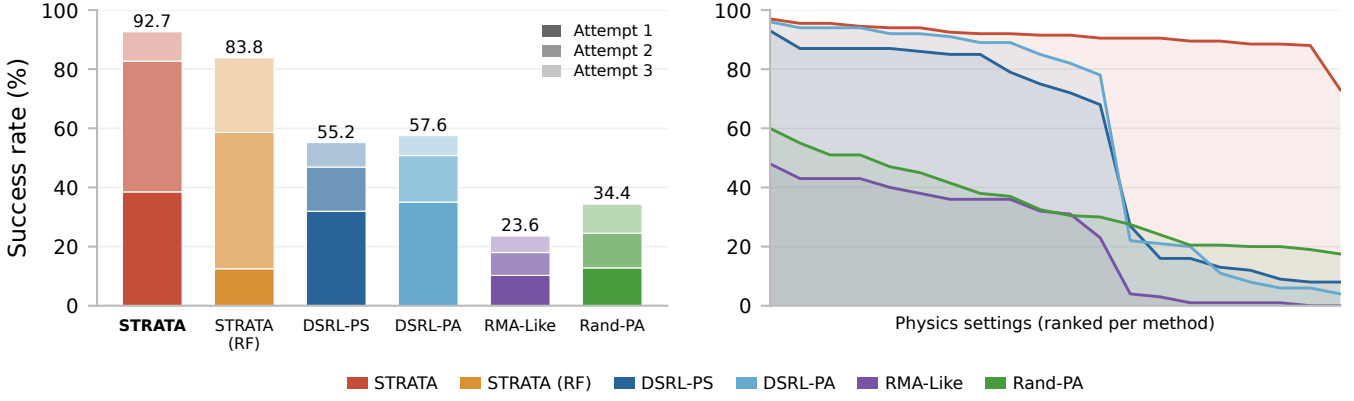


Fig. 4. **Success across attempts and physical conditions.** **Left:** segments show the fractions of trials that first succeed on attempts 1, 2, and 3; cumulative heights give Success@1, Success@2, and Success@3. **STRATA** (RF) randomizes the first strategy and uses learned selection thereafter. Rand-PA selects randomly before every attempt. **Right:** Success@3 across the 20 conditions, sorted separately for each method; the same rank can refer to different conditions. The random-first variant is shown only on the left. Results average two seeds, with 100 trials per condition and seed.

selection, although it changes both the high-level action space and the training of the executor.

The RMA-like baseline reaches 23.6% Success@3. Repeated selection permits strategy changes within an attempt and creates many high-level decisions before the task outcome is known. Selecting once keeps the chosen strategy consistent across predictions and gives RL one choice per attempt to evaluate. This comparison favors attempt-level selection, but does not distinguish the effects of within-attempt switching from the increased learning burden. Per-attempt DSRL also exceeds per-step DSRL (57.6% versus 55.2%), yet remains well below **STRATA**. Holding a choice fixed is therefore not enough in this comparison: what that choice represents also matters. Section IV-C examines this representation directly.

We also compare random selection with the same frozen execution policy, codebooks, and three-attempt budget. Random strategies are sampled uniformly from all codebook-index pairs. Random selection on every attempt (Rand-PA) reaches 34.4% Success@3. When only the first strategy is random and the learned policy selects the retries (**STRATA** RF), success reaches 83.8%. Thus, learned retry selection remains effective even after an interaction that the policy did not choose. This recovery shows that the benefit is not confined to learning a good initial strategy. Section IV-D tests how much later choices benefit from observing the interaction beyond remembering which codes failed.

C. Strategy Representation

A useful strategy representation should capture demonstrated strategies, support their execution, and let the high-level policy learn which to select. We compare residual VQ with fully continuous conditioning, single VQ, VQ with a continuous residual, and parallel VQ. The fully continuous variant uses the encoder’s 64-dimensional representation without quantization. Single VQ uses one 32-entry codebook. The hybrid concatenates a selected 64-dimensional code with a two-dimensional continuous residual and projects the result back to 64 dimensions. Parallel VQ splits the 64-dimensional

embedding into two halves, quantizes each with a separate 32-entry codebook, and concatenates the selected entries. Residual VQ instead quantizes the full embedding and then its residual, summing two 64-dimensional entries. Both two-code variants allow 1,024 index pairs, although their entry dimensions differ.

To examine which behaviors the frozen controllers can execute without learned selection, we construct a demonstration-based reference. We first screen 50 source motions by motion replay, using 100 rollouts per motion and physical condition. This separate screen selects a broadly effective motion g and a condition-specific motion b_c for each of the 20 evaluation conditions. These choices are fixed before evaluating the flow-matching controllers.

For each reset, we retrieve a training demonstration of the selected motion with a similar initial box pose, encode it with the representation being evaluated, and run the frozen controller. We evaluate each selection rule with 100 rollouts per condition. Let $p_{g,c}$ and $p_{b_c,c}$ denote their single-attempt success rates in condition c . We estimate a hypothetical three-attempt trial with g first and b_c on two retries:

$$\hat{S}_3 = \frac{1}{20} \sum_{c=1}^{20} [1 - (1 - p_{g,c})(1 - p_{b_c,c})^2]. \quad (8)$$

The estimate assumes independent attempts. Because this reference uses the known physical condition and privileged pose information to choose demonstrations, it is used only to analyze execution under demonstration-based selection. It is limited by the searched demonstrations and is not an upper bound on learned strategy selection.

Figure 5 shows why execution and learned selection must both be considered. Fully continuous conditioning reaches 74.7% in the demonstration-based estimate but only 51.3% with learned selection. The controller can therefore execute useful behaviors that the learned selector does not access as reliably. This gap points to difficulty learning choices in the tested continuous space, rather than an inability to execute

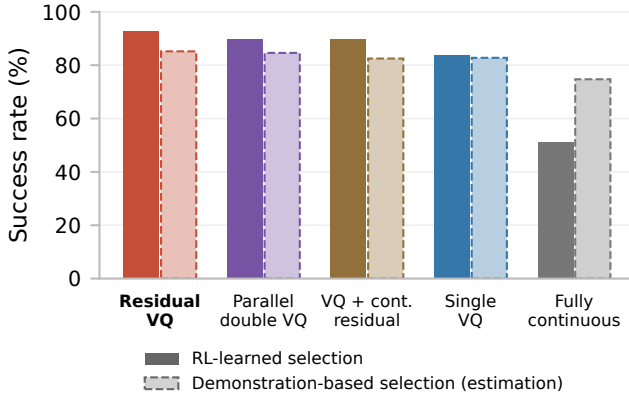


Fig. 5. **Strategy representations for execution and selection.** Light bars show the demonstration-based three-attempt estimate from Eq. 8; dark bars show Success@3 with RL-learned selection. The reference selects source motions using known physical conditions and retrieves demonstrations using object pose. It is restricted to these candidates and is not an upper bound. All representations with a discrete component support strong learned selection. Residual VQ has the highest mean at 92.7%, with parallel VQ and the hybrid both reaching 90.0%.

any useful strategy. The hybrid reaches 82.5% in the reference and 90.0% with learned selection: continuous variation can work well when paired with discrete choices.

Single VQ reaches 82.8% in the reference and 83.8% with learned selection. Both two-code variants improve on it, but also increase the number of possible choices from 32 to 1,024, so this comparison changes capacity as well as structure. Parallel VQ reaches 84.6% and 90.0%, while residual VQ reaches 85.2% and 92.7%. Residual VQ has the highest mean under both measures, with parallel VQ and the hybrid close behind in learned selection. Together, these results favor representations with a discrete component under our learning setup, including those that retain continuous variation. Learned selection can exceed the reference because the two use different selection procedures: RL chooses codes from interaction history, whereas the reference uses a restricted set of source motions and pose-based demonstration retrieval.

We next vary how the strategy representation is trained. The reconstruction variant adds a unit-weighted mean-squared error for reconstructing the normalized complete joint-target sequence. The pretraining variants first learn the representation with reconstruction and VQ losses, then either freeze it or update it jointly with the flow-matching policy. Each resulting controller is frozen before high-level RL training. Joint training without reconstruction reaches the highest mean Success@3, 92.7% (Table I). Fine-tuning the pretrained representation reaches 91.6%, freezing it reaches 90.1%, and adding reconstruction during joint training reaches 89.3%. Joint training through action prediction thus gives the best result without a separate pretraining stage. The pretraining variants use additional training, so this is not an equal-compute comparison.

Training	Success@3 (%)
Joint, without reconstruction (ours)	92.70
Joint, with reconstruction	89.28
Representation pretrained and frozen	90.13
Representation pretrained and fine-tuned	91.63

TABLE I. **Effect of representation training on task success.** Success@3 after high-level RL training with each representation and its frozen controller. The results reflect both execution and learned selection, rather than demonstration-based estimates. Joint training without reconstruction gives the highest mean.

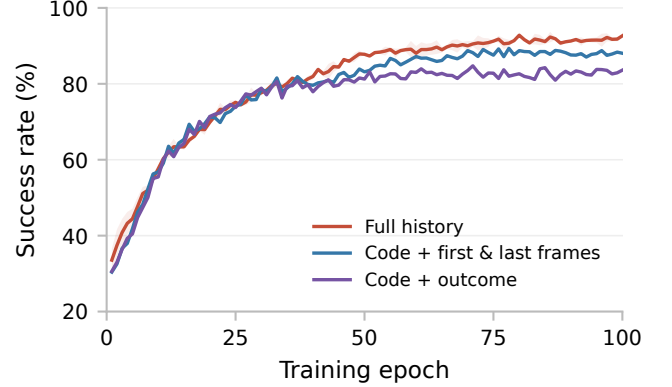


Fig. 6. **Information from previous attempts.** Success@3 during high-level training with full observation–action histories, strategy codes with initial and final observations (“Code + first & last frames”), or strategy codes alone (“Code + outcome”). Every completed attempt available for retry selection has failed, so its outcome flag adds no information. Full history gives the highest reported success.

D. Information from Previous Attempts

We next ask what information from a failed attempt helps the high-level policy choose the next strategy. Remembering the strategy code identifies what has failed, but not how the object responded. We compare full observation–action histories with two summaries. The first supplies the previous strategy code with the initial and final observations (“Code + first & last frames” in the plot). The second retains only the previous strategy code (“Code + outcome”). Since a retry is selected only after failure, the outcome flag is constant and adds no information to that code.

Full history reaches 92.7% Success@3, compared with 88.1% using the code with initial and final observations and 83.6% using the code alone (Fig. 6). Remembering failed strategies already supports useful retry choices, but observing the scene before and after an attempt improves success further. Full history performs best, supporting the value of recording how the robot and object interact rather than only which strategy failed. Because it contains both intermediate observations and commands, this comparison does not isolate either as the sole source of the improvement.

E. Real-Robot Evaluation

Finally, we test zero-shot transfer to a real robot. We deploy the simulation-trained policies on a TIAGo robot without real-world training or fine-tuning. We evaluate four conditions: a

Condition	Ours	DSRL
Standard	90	90
Smooth surface, lightweight object	90	50
Heavy lid	60	60
High hinge damping	40	20
Average	70	55

TABLE II. **Zero-shot real-robot performance across physical conditions.** Success@3 (%) over 10 trials per condition and method. **STRATA** matches per-attempt DSRL on the standard and heavy-lid conditions and improves success on the smooth-surface and high-damping conditions, averaging 70% versus 55%.

standard setup, a smooth surface with a lightweight object, a heavy lid, and increased hinge damping. Figure 3 shows example strategies and outcomes.

We compare **STRATA** with history-conditioned, per-attempt DSRL using up to three attempts per trial and 10 trials per condition and method. We reconstruct point clouds from stereo images with Fast-FoundationStereo [34] and remove the table plane. These point clouds and robot proprioception provide the policy observations.

STRATA achieves 70% Success@3, compared with 55% for per-attempt DSRL (Table II). First-attempt success is similar, at 20% and 22.5%, but Success@2 reaches 52.5% and 40%, respectively. By the third attempt, **STRATA** leads by 15 percentage points. It improves success under the smooth-surface and high-damping conditions and matches DSRL under the standard and heavy-lid conditions. These trials show that the simulation-trained selector remains useful for choosing retries from real interactions without further training.

V. CONCLUSION

We presented **STRATA**, a framework that connects consistent execution within attempts with strategy selection across attempts through a learned strategy representation. Successful demonstrations jointly train this representation and a strategy-conditioned flow-matching policy. Each code represents a strategy across a complete demonstration and stays fixed throughout an attempt, while the controller responds to current observations. A high-level policy then uses these codes as its action space and learns from task rewards to select strategies using previous attempts’ interaction histories. Demonstrations thus teach how to execute different strategies, while interaction teaches which one to try next. In simulated box opening under 20 physical conditions, **STRATA** achieves 92.7% success within three attempts, compared with 57.6% for per-attempt DSRL. Zero-shot deployment on a real robot reaches 70%, compared with 55% for per-attempt DSRL.

A strategy representation must support both execution and selection: it needs enough detail to preserve distinct demonstrated behaviors and a structure that lets the high-level policy learn which ones to try. Our representation study shows that two discrete codes serve both purposes, with residual VQ achieving the highest success among the evaluated representations. Learned selection substantially outperforms random choices from the same strategy space, and learning the first choice improves success further. Our evaluation focuses

on box opening under a fixed set of physical conditions, and our formulation assumes object resets between attempts. Future work will examine other manipulation tasks, retries without resets, and ways to expand the strategy repertoire when demonstrated behaviors are insufficient.

VI. LIMITATIONS

STRATA can only select strategies that are supported by the demonstration-trained execution policy. If the available strategies are insufficient for a physical condition, better selection alone cannot solve the task. Extending the framework to discover or refine new strategies through interaction is an important direction for future work.

Our current formulation assumes that the object can be reset between attempts while its physical properties remain unchanged. This simplifies the retry problem by removing changes caused by previous failures. Future work should study recovery without such resets, where the robot must continue from the state left by an unsuccessful attempt.

Finally, we evaluate **STRATA** on box opening under a range of physical conditions. These experiments test strategy selection under hidden physical variation, but do not establish that the same gains will hold across manipulation tasks more broadly. We plan to evaluate the framework on tasks with different strategy structures and sources of physical uncertainty.

REFERENCES

- [1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [2] E. Chisari, N. Heppert, M. Argus, T. Welschehold, T. Brox, and A. Valada, “Learning robotic manipulation policies from point clouds with conditional flow matching,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 982–993. [Online]. Available: <https://proceedings.mlr.press/v270/chisari25a.html>
- [3] Y. Wang, X. Zhang, R. Wu, Y. Li, Y. Shen, M. Wu, Z. He, Y. Wang, and H. Dong, “AdaManip: Adaptive articulated object manipulation environments and policy learning,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=Luss2sa0vc>
- [4] M. Du, A. Khazatsky, T. Gerstenberg, and C. Finn, “To err is robotic: Rapid value-based trial-and-error during deployment,” *arXiv preprint arXiv:2406.15917*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.15917>
- [5] A. Razmjoo, S. Calinon, M. Gienger, and F. Zhang, “CCDP: Composition of conditional diffusion policies with guided sampling,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 20 036–20 043.
- [6] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 258–282. [Online]. Available: <https://proceedings.mlr.press/v305/wagenmaker25a.html>
- [7] X. Lei, M. Wang, W. Zhou, X. Lu, and H. Li, “Primary-fine decoupling for action generation in robotic imitation,” *arXiv preprint arXiv:2602.21684*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.21684>
- [8] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, “Behavior generation with latent actions,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 26 991–27 008. [Online]. Available: <https://proceedings.mlr.press/v235/lee24y.html>

- [9] Z. Zhao, M. Leggiero, Y. Chen, H. Liu, Y. Wu, H. Xue, S. Zhan, and Y. Zhao, "VQActFlow: Vector-quantized action mode steering for multi-task robot manipulation," *arXiv preprint arXiv:2606.21600*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.21600>
- [10] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning latent plans from play," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 1113–1132. [Online]. Available: <https://proceedings.mlr.press/v100/lynch20a.html>
- [11] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar, "MimicPlay: Long-horizon imitation learning by watching human play," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 201–221. [Online]. Available: <https://proceedings.mlr.press/v229/wang23a.html>
- [12] K. Pertsch, Y. Lee, and J. Lim, "Accelerating reinforcement learning with learned skill priors," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 188–204. [Online]. Available: <https://proceedings.mlr.press/v155/pertsch21a.html>
- [13] A. Ajay, A. Kumar, P. Agrawal, S. Levine, and O. Nachum, "OPAL: Offline primitive discovery for accelerating offline reinforcement learning," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=V69LGwJ0IIN>
- [14] E. Rosete-Beas, O. Mees, G. Kalweit, J. Boedecker, and W. Burgard, "Latent plans for task-agnostic offline reinforcement learning," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1838–1849. [Online]. Available: <https://proceedings.mlr.press/v205/rosete-beas23a.html>
- [15] J. Hu, Z. Wang, P. Stone, and R. Martín-Martín, "Disentangled unsupervised skill discovery for efficient hierarchical reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/8c263f70550cc7d69dba3fc170a23e77-Abstract-Conference.html
- [16] J. Hu, P. Stone, and R. Martín-Martín, "SLAC: Simulation-pretrained latent action space for whole-body real-world RL," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2966–2982. [Online]. Available: <https://proceedings.mlr.press/v305/hu25b.html>
- [17] T. Nam, S.-H. Sun, K. Pertsch, S. J. Hwang, and J. J. Lim, "Skill-based meta-reinforcement learning," in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=jELW-Fh9bV>
- [18] A. Tang, W. Chen, A. Wagenmaker, C. Finn, and S. Levine, "Improving robotic generalist policies via flow reversal steering," *arXiv preprint arXiv:2606.13675*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.13675>
- [19] R. Martín-Martín and O. Brock, "Coupled recursive estimation for online interactive perception of articulated objects," *The International Journal of Robotics Research*, vol. 41, no. 8, pp. 741–777, 2014.
- [20] S. Dass, J. Hu, B. Abbatematteo, P. Stone, and R. Martín-Martín, "Learning to look: Seeking information for decision making via policy factorization," in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 4425–4445. [Online]. Available: <https://proceedings.mlr.press/v270/dass25a.html>
- [21] R. Bajcsy, "Active perception," *Proceedings of the IEEE*, vol. 76, no. 8, pp. 996–1005, 1988.
- [22] J. Bohg, K. Hausman, B. Sankaran, O. Brock, D. Kragic, S. Schaal, and G. S. Sukhatme, "Interactive perception: Leveraging action in perception and perception in action," *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1273–1291, 2017.
- [23] R. K. Mehra, "Optimal input signals for parameter estimation in dynamic systems—survey and new results," *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 753–768, 1974.
- [24] Y. Gao, J. J. Liu, S. Li, and S. Song, "Gated memory policy: In-context memorization and adaptation," *arXiv preprint arXiv:2604.18933*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.18933>
- [25] K. Wang, S. Yeom, J. Cao, Y. Zhi, N. Shinde, and M. C. Yip, "Remember what you did?: Learning behavioral memories for partially observable object manipulation," *arXiv preprint arXiv:2606.21188*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.21188>
- [26] R. Shah, Y. Li, F. Bello, Y. Zhu, and R. Martín-Martín, "Memory retrieval in visuomotor policies for long-horizon robot control," in *Proceedings of Robotics: Science and Systems*, 2026. [Online]. Available: <https://www.roboticsproceedings.org/rss22/p010.html>
- [27] J. Liu, J. Li, Z. Zhuang, H. Yu, Q. Chen, L. Cao, Y. Lu, R. Chen, Y. Cao, C. Zhang, Y. Lin, and M. Xu, "What do VLAs actually learn through in-context failure conditioning?" in *2nd 3D-LLM/VLA Workshop at CVPR*, 2026. [Online]. Available: <https://github.com/jedward225/RoboRetry/>
- [28] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Proceedings of Robotics: Science and Systems*, 2021.
- [29] A. Xie, L. M. Bhamidipaty, E. Z. Liu, J. Hong, S. Levine, and C. Finn, "Learning to explore in POMDPs with informational rewards," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 54414–54429. [Online]. Available: <https://proceedings.mlr.press/v235/xie24a.html>
- [30] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, "RL²: Fast reinforcement learning via slow reinforcement learning," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=HkLXCE9lx>
- [31] J. Bauer, K. Baumli, F. Behbahani, A. Bhoopchand, N. Bradley-Schmieg, M. Chang, N. Clay, A. Collister, V. Dasagi, L. Gonzalez, K. Gregor, E. Hughes, S. Kashem, M. Loks-Thompson, H. Openshaw, J. Parker-Holder, S. Pathak, N. Perez-Nieves, N. Rakicevic, T. Rocktäschel, Y. Schroecker, S. Singh, J. Sygnowski, K. Tuyls, S. York, A. Zacherl, and L. M. Zhang, "Human-timescale adaptation in an open-ended task space," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 1887–1935. [Online]. Available: <https://proceedings.mlr.press/v202/bauer23a.html>
- [32] Y. Li, X. Mao, Y. Yuan, K. Sim, B. Eisner, and D. Held, "Learn from what we HAVE: History-aware VErifier that reasons about past interactions online," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2063–2092. [Online]. Available: <https://proceedings.mlr.press/v305/li25e.html>
- [33] H. Hao, S. N. Syed, J. Ichnowski, and J. Schneider, "FAR: Failure-aware retry for test-time recovery and continual policy improvement," *arXiv preprint arXiv:2607.01111*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01111>
- [34] B. Wen, S. Dewan, and S. Birchfield, "Fast-foundationstereo: Real-time zero-shot stereo matching," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2026, pp. 7513–7524.